

An AI agent that cannot overspend, because the ledger will not let it.

A Daml mandate on Canton holds the budget, the approved payee, the expiry and the revocation. The agent submits requests. Canton decides.

Contents
• Twelve cases, each with the operation performed and the value returned • The line of Daml that produces each refusal • Failure modes on a shared testnet, and how each is handled

Verified on
Ledger <code>api.validator.dev.digik.cantor8.tech</code> Package <code>proj-0.0.1 · 373262fc...</code> Parties <code>3 hosted, act-as granted</code> Balance <code>5.0000 Amulet</code> Tests <code>13 Daml scripts passing</code>

The usual answer is to hand the agent a wallet key.

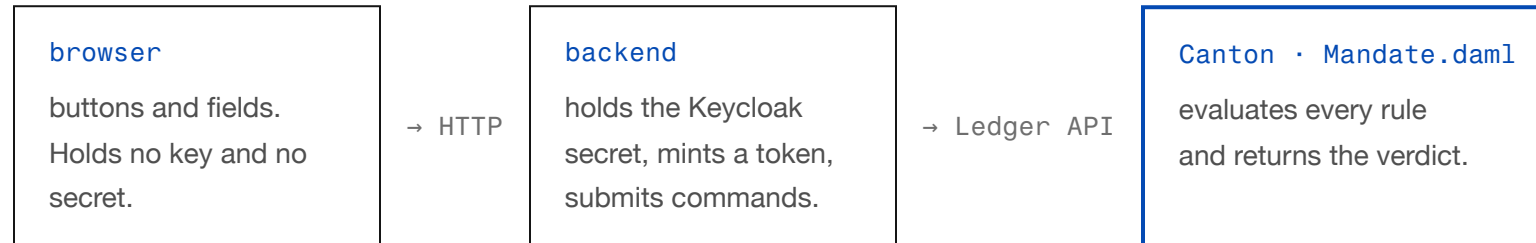
If the key is in the agent's hands, the only thing between a manipulated model and the money is the model's own judgement. In May 2026 a Morse-code-encoded instruction persuaded an AI trading bot to drain about \$150k: the safety layer decoded the message, then obeyed it.

So the limit cannot live in the model. Nor in our backend, because anyone reaching the ledger directly would walk around it. It lives in the contract.

Prior work in this area
- World's First Private AI Agent Payment Settles on Canton Network, April 2026 \$200k awarded to a Cambridge researcher working on agentic AI on Canton, March 2026

Four questions a mandate answers
how much, ever lifetime cap
how much, how fast period cap
paid to whom counterparty allow-list
until when, and can it be stopped expiry and revocation

The browser asks, the backend submits, Canton decides.



Why the backend exists at all

Two reasons only: the client-credentials secret cannot ship to a browser, and every ledger call needs a fresh bearer token. It adds no rules of its own. Remove it and the caps still hold.

The same path for a language model

An MCP server exposes `pay`, `revoke` and `state` to an LLM. Those tools call the same dispatch as the buttons, so an LLM talked into overspending is refused by Canton, not by a check in our code.

Seven rules, each one line, all inside the Charge choice.

```
-- both parties must consent for the mandate to exist
signatory owner, spender

choice Charge : ContractId Mandate
  with amount : Decimal
       payee   : Party
  controller spender      -- the agent acts alone
  do
    now <- getTime
    assertMsg "mandate expired"          (now < expiresAt)
    assertMsg "amount must be positive"  (amount > 0.0)
    assertMsg "mandate revoked"          (not revoked)
    assertMsg "charge would exceed the period cap"
      (curPeriodSpent + amount <= periodCap)
    assertMsg "charge would exceed the cap"
      (spent + amount <= cap)
    assertMsg "payee not in allow-list"
      (payee `elem` counterparties)
```

Why this placement matters

`controller spender` means the agent's authority alone drives a charge, which is the point: no human signature per purchase. The assertions sit in the choice body, so they bind anyone exercising it. The quoted strings are exactly what the browser displays, because Canton returns them as `AssertionFailed`.

Revocation, separately

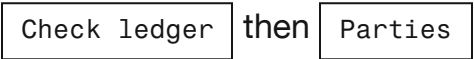
```
choice Revoke : ContractId Mandate
  controller owner      -- owner alone; agent cannot block
```

Also enforced

`ensure` holds `spent <= cap` and `periodSpent <= periodCap` as invariants on every state transition, so a mandate breaching them cannot exist at all.

Confirming the ledger is live before anything is claimed of it.

operation



what the backend does

```
POST /realms/master/.../token → bearer
GET /v2/state/ledger-end
```

expect in the top-right

Canton DevNet · offset 2 922 xxx

expected

offset increases between presses

The offset is the live ledger position and climbs as the shared network commits transactions. A frozen number would mean a cached response.

Parties · expected values		
Name	Role	Amulet
Business	whose money it is	0.0000
AI agent	spends on their behalf	5.0000
Cloud vendor	approved supplier	0.0000

failure mode

One of DevNet's load-balancer addresses black-holes traffic, so a read occasionally hangs. Reads use a 3 second timeout and retry up to five times, so this resolves itself in about a second. If the panel says *balance unknown* that is a failed query, not a zero balance. The two are reported differently on purpose.

Propose and accept, so neither party is bound without consenting.

fields

Lifetime cap 500 · Period cap 100 · Period hours 24
Approved payee Cloud vendor · Expires in 720

operation

Grant mandate

two ledger transactions

Business creates a `MandateProposal`; the agent exercises `Accept`, which creates the `Mandate`.

expected · Live mandate panel

AI agent spending for Business

lifetime 0.00 of 500.00 · period 0.00 of 100.00 per 24h

Why two steps and not one

The `Mandate` has two signatories, so it cannot be created by the owner alone, because Daml will not let one party impose an obligation on another. The proposal carries only the owner's signature; accepting adds the agent's. This is the standard shape for any agreement on Canton.

failure mode

Writes are never retried automatically. DevNet returns 503 while still forwarding the command, so a retry races the original, and Canton then answers `SUBMISSION_ALREADY_IN_FLIGHT`, after which every contract id we hold is stale. Instead the error surfaces and you press the button again, which is safe because each submission carries a fresh command id.

One atomic transaction updates the budget and writes the audit record.

fields
Amount 25 · Payee Cloud vendor

operation

Charge

expected

Charge approved: 25 to Cloud vendor

What Canton does in that one transaction

- Archives the current `Mandate`
- Creates its successor with `spent` and `periodSpent` increased
- Creates a `MandateChargeAudit` recording amount, payee, time and the rule that allowed it

All three or none. There is no state in which the budget moved but the audit record is missing.

Live mandate · after

lifetime 25.00 of 500.00
period 25.00 of 100.00 per 24h
meter tracks whichever budget is tighter

Audit trail · new row

Amount	Payee	Result
25.00	Cloud vendor	accepted

scope

Charging 25 authorises on the ledger but cannot settle, because the agent holds only 5 Amulet. The response says `authorised: true, settled: false` with the reason, rather than implying money moved. Settlement is shown separately at 0.1.

A charge larger than the remaining daily budget is refused.

fields

Amount 999 · Payee Cloud vendor

operation

Charge

expected · refused by Canton

charge would exceed the period cap

Same outcome for 100 000, and for any amount above the remaining daily budget. The mandate is untouched: `spent` stays at 25 and no audit record of a successful charge is created. The refusal itself is recorded.

```
assertMsg "charge would exceed the period cap"
          (curPeriodSpent + amount <= periodCap)
```

Read the daily cap as a rate limit

A lifetime cap alone lets a hijacked agent drain everything in one minute. The period cap bounds the loss per day, which is what buys a human time to notice and revoke.

window semantics

Nothing in Daml runs on a schedule. `Charge` rolls the window forward when it next runs, advancing by whole periods from `periodStart` and never to `now`. Anchoring to `now` would let the agent slide the window by timing its charges.

The lifetime rule is only reachable when the daily cap sits above it.

The period cap is checked first, so on a large single charge it always fires and the lifetime message is unreachable. Grant a mandate whose period cap sits above its lifetime cap, and the lifetime rule becomes the binding one.

```
grant
Lifetime cap 100 · Period cap 1000
subsequent charge
90 → accepted, then 20
```

expected · refused by Canton
charge would exceed the cap

Order of evaluation
1 mandate expired
2 amount must be positive
3 mandate revoked
4 period cap ← fires first
5 lifetime cap
6 payee allow-list

The message names the first rule that failed, not every rule that would have. Evaluation short-circuits at the first assertion to break.

```
arithmetic
90 + 20 = 110 > 100 lifetime
90 + 20 = 110 < 1000 period
```

A payment within budget to a party outside the allow-list is refused.

fields

Amount 5 · Payee Unapproved vendor

operation

Charge

expected · refused by Canton

payee not in allow-list

The amount is well within both budgets. Only the recipient is wrong. This is the exfiltration case: a manipulated agent sending a small, plausible payment somewhere it was never authorised to send.

```
assertMsg "payee not in allow-list"
         (payee `elem` counterparties)
```

Party resolution

Unapproved vendor resolves to a real party the node hosts, not a malformed name. So the refusal is genuinely the allow-list rejecting an existing party, rather than Canton rejecting an invalid party id. An arbitrary string is refused too, but as a malformed party rather than an unapproved one.

related case

Amount -50 to the approved vendor returns amount must be positive. A negative charge would otherwise run the balance backwards and manufacture headroom.

The owner revokes alone, and the agent cannot delay it.

operation

Revoke now

expect

Live mandate shows **REVOKED**; top rail reads *mandate revoked*

subsequent charge

Amount 5 · Payee Cloud vendor

expected · refused by Canton

mandate revoked

A charge that would have been perfectly valid a second earlier is now refused. Nothing about the amount or the payee changed.

```
choice Revoke : ContractId Mandate
  controller owner
  do
    now <- getTime
    newCid <- create this with revoked = True
    create MandateRevokeAudit with
      owner; spender; at = now
      reason = "owner revocation"
    pure newCid
```

Why a flag rather than simply archiving

Archiving would also stop the agent, since the contract id would no longer exist. Keeping a revoked mandate on the ledger means the owner can still read its final state and tie the revocation event to the record it applies to. Both routes stop the spending; this one leaves better evidence.

`Adjust` cannot undo it either: it carries the existing fields forward, so `revoked` stays true, and it needs both signatures regardless.

Seven purchases chosen by the agent, three of them wrong.

operation



Expected outcome, item by item		
Amount	Result	Reason from the ledger
30.00	settled	—
15.00	settled	—
450.00	declined	period cap
20.00	declined	not in allow-list
25.00	settled	—
-40.00	declined	must be positive
20.00	settled	—

The agent does not know which three are wrong. It attempts all seven.

7

attempted

510.00

overspend prevented

3

refused on the ledger

90.00

settled within mandate

100%

verdicts matching a correct implementation

precondition

The backlog needs about 90 of the 100 daily budget. Run it against a part-spent mandate and a legitimate purchase is refused for lack of headroom, giving 3 settled and 4 refused. The app now refuses to start and tells you how much budget is left rather than producing a misleading score.

Every action, and the permission that allowed it.

operation

Statement

, with the audit trail alongside

expect

One row per attempt, accepted and refused alike, each with the deciding rule. Row count matches attempts made since the mandate was granted.

```
AGENT SPENDING STATEMENT
Mandate holder  AI agent acting for Business
Lifetime budget 90.00 of 500.00 used
Period budget   90.00 of 100.00 used (resets every 24h)
Approved payees Cloud vendor

Reserved capacity, annual commitment  450.00  declined
      reason: charge would exceed the period cap
Refund of a duplicate invoice          -40.00  declined
      reason: amount must be positive
Prevented                               510.00  3 declined
```

These records are contracts

MandateChargeAudit is signed by the spender and observed by the owner; MandateRevokeAudit the other way around. Because they are on the ledger they cannot be edited after the fact, which is the property an auditor actually cares about.

scoping

Audit contracts accumulate across every mandate this owner has ever held, and testing left 113 of them. The view is scoped to records created since the mandate in view was granted, so a fresh session starts clean. The earlier records remain on the ledger.

Authorisation and settlement are two transactions, in that order.

operation

Pay 0.1 for real

expect · balances

AI agent 5.0000 → 4.9000 -0.1
Cloud vendor 0.0000 → 0.1000 +0.1

reset

Return the coin

 sweeps the balance back, so the case is repeatable

expected · settlement detail

transferKind: offer · accepted: true

A transfer arrives as `direct` when the receiver holds a live pre-approval and as an `offer` otherwise. An offer does not move the receiver's balance until accepted, so we accept on the payee's behalf and the money genuinely lands.

Why Charge does not move the money itself

A token-standard transfer needs disclosed contracts fetched from the issuer's registry for that single transaction. A Daml choice body cannot make an HTTP call to go and get them. So the agent authorises on the ledger first, and only an authorised charge reaches the settlement code at all.

scale

The agent holds about 5 Amulet while its authorisation cap is 500. The cap is therefore demonstrated by refusal, not by exhausting the balance. Each settlement run costs roughly 0.1 in network fees, and both parties belong to this project, so no value leaves it.

Stopping the backend loses no state, because the application never held it.

operation

SIGINT the backend, restart it, reload the page

expect

The same mandate, with the same `spent`, and charging continues from there.

measured

before kill spent 90.00 of 500.00
after restart spent 90.00 of 500.00

The backend keeps only a contract id in memory. On startup it queries the active contract set and adopts the live mandate again. Nothing about the budget was ever stored in the application.

A related refusal

`Contract could not be found with id 00...`

Every accepted charge archives the mandate and creates a successor, so a second browser tab, or a script run in a terminal, leaves the first view holding a superseded id. The backend detects this, re-reads the current mandate and retries once, so pressing the button again works. It is Canton being strict about contract identity, not a failure.

retry policy

Reads are retried, five attempts at a 3 second timeout each. Writes are never retried, because a stalled submission may still be in flight. The heavy `/v2/parties` read gets the full 30 seconds, since it returns ten thousand records a page.

Each failure has an identified cause and a defined recovery.

Message	Cause	Recovery
HTTP 503 ... /v2/commands/	DevNet refused a write under load	Resubmit. Each submission carries a fresh command id.
SUBMISSION_ALREADY_IN_FLIGHT	The same command was submitted twice	Wait a moment, then read state. One of them committed.
Contract could not be found	Contract id superseded by a charge elsewhere	Resubmit; the backend re-reads and retries once.
read operation timed out	A black-holed load-balancer address	Resolves itself. Retries are automatic on reads.
balance unknown	Balance query failed	Not a zero balance. Re-query.
only 4.20 of budget left ...	Agent backlog needs about 90	Grant a fresh mandate, then run the agent.
sender has no spendable Amulet	Authorised, but nothing to settle with	Return the coin, or ask Cantor8 to fund the party.
mandate revoked	Working as designed	Grant a new mandate to continue.

None of these represent the wallet failing. A failure would be a charge succeeding when a rule should have refused it, and that outcome did not occur in any of the twelve cases.

The tools a language model drives reach the ledger by the same path.

An MCP server exposes the wallet as tools over stdio JSON-RPC. Early on those tools called the backend's simulation helpers rather than the live ledger, which would have meant a model was policed by Python while the system claimed Canton was doing it. They now go through one shared dispatch, so the HTTP buttons, the autonomous agent and the LLM all take the same route.

Calls a manipulated model might make	
Tool call	Verdict
pay(25, Cloud vendor)	allowed
pay(9999, Cloud vendor)	period cap
pay(10, Unapproved vendor)	not in allow-list
pay(-5, Cloud vendor)	must be positive

The claim this supports

A prompt can talk a model into attempting any of the above. None of it succeeds. The boundary is not in the model's reasoning, nor in the tool layer, nor in our HTTP handler. It is a rule in a contract the model cannot reach.

prior art

Google's Agent Payments Protocol calls its primitive a *mandate*: a signed grant carrying constraints for when the human is not present. Ours is the same idea with the constraints executed by the ledger rather than checked by a verifier, so authorisation and settlement land in one signed transaction.

What the implementation does, and where it stops.

Live on Canton DevNet

- The mandate is a contract with a real contract id
- Lifetime cap, period cap, allow-list, expiry and revocation are enforced in `Mandate.daml`
- Every refusal quoted in this deck is an `AssertionFailed` returned by Canton
- Charge and revoke audit records are contracts on the ledger
- Canton Coin moves and the payee's balance changes
- Killing the backend loses no state
- The MCP tools take the same path as the UI

Not done

- Settlement moves 0.1 at a time against a balance of about 5, so the cap is proven by refusal rather than exhaustion
- The allow-list is fixed when the mandate is granted; changing it needs `Adjust` and both signatures
- Period windows roll forward on the next charge, not on a timer
- `Charge` does not itself transfer funds, for the registry reason given earlier
- Tests share a package with the templates, which would bloat a production participant

The honest gap is settlement scale, not enforcement. Every limit is real; the pot of money behind it is small.

What the twelve cases produced.

12
test cases, all passing

13
Daml scripts passing

510.00
overspend prevented in one agent run

100%
verdicts correct

0
charges that got through when they should not have

6
distinct rules observed refusing

Rules observed refusing, by name	
Rule	Triggered by
period cap	999, 100000, 450
lifetime cap	90 then 20 against cap 100
allow-list	Unapproved vendor
positive amount	-50, -40
revoked	any charge after Revoke
expiry	covered by Daml script rather than interactively

Expiry is easier to prove in a script than interactively, since it needs the ledger clock moved forward. `daml test` does that in memory.

The agent may attempt anything. The ledger decides.

The business grants a budget, a daily limit, one approved supplier and instant revocation. Those limits are a contract on Canton rather than a check in the agent's reasoning or in our server, so no prompt can talk past them.

Where to look

```
proj/daml/Mandate.daml the rules
proj/daml/*Test*.daml 13 passing scripts
demo/live.py real ledger calls
demo/agent.py the autonomous agent
DEMO.md this walkthrough as a runbook
```

Run it

```
git pull
./run.sh
then localhost:8000 on your own machine
```